

Universidad Técnica Nacional
Sede del Pacífico

Bachillerato en Tecnologías de Información

Curso:

Base de Datos Avanzada

Proyecto:

Transporte Público Inteligente

Estudiantes:

Alvarez Alvarado Jorjan Mauricio | 208740822

Amores Barrantes Esteban David | 208630913

Campos Torres Marco Antonio | 604990267

Montero Carvajal Laura Vanessa | 604840637

Puntarenas, 2026

1. Introducción y Descripción del Escenario

La municipalidad requiere monitorear rutas, unidades y tiempos de viaje del transporte público. Actualmente no es posible medir atrasos ni optimizar los recorridos utilizando datos históricos. Este proyecto propone una solución on-premise que integra distintos motores de base de datos para centralizar la información operativa, capturar telemetría en tiempo real y generar análisis histórico de movilidad.

Alcance: Registro de rutas, choferes, horarios y unidades; captura de eventos GPS/telemetría; generación de reportes oficiales; consolidación analítica para la toma de decisiones municipales.

Motores de base de datos: PostgreSQL (datos maestros y transaccionales), MongoDB (eventos GPS y telemetría), SQL Server (Data Warehouse / análisis histórico de movilidad).

Formatos adicionales: JSON para configuración de sensores; XML para reportes oficiales.

Integración: Procesamiento de geolocalización mediante ETL, con investigación aplicada de Apache Kafka como mecanismo de mensajería para la ingesta de eventos.

2. Documento de Requerimientos

2.1 Requisitos Funcionales

ID	Nombre	Descripción
RF-01	Gestión de rutas	El sistema debe permitir registrar, editar, consultar y desactivar rutas de transporte público (origen, destino, paradas intermedias, distancia y tiempo estimado).
RF-02	Gestión de choferes	El sistema debe permitir registrar y administrar los datos de los choferes (identificación, licencia, categoría, estado laboral, historial de asignaciones).
RF-03	Asignación chofer-unidad-ruta	El sistema debe permitir asignar choferes a unidades y rutas específicas, evitando traslapes de horario para un mismo chofer.
RF-04	Gestión de unidades	El sistema debe permitir registrar y administrar las unidades de transporte (placa, capacidad, modelo, año, estado operativo).
RF-05	Gestión de horarios	El sistema debe permitir crear y modificar horarios por ruta, incluyendo frecuencia, horas pico y días de operación.
RF-06	Configuración de sensores (JSON)	El sistema debe almacenar y actualizar la configuración de los sensores instalados en cada unidad (tipo, frecuencia de muestreo, umbrales) en formato JSON dentro de PostgreSQL.

RF-07	Reportes oficiales (XML)	El sistema debe generar y almacenar reportes oficiales de operación en formato XML conforme al estándar solicitado por la municipalidad.
RF-08	Captura de eventos GPS	El sistema debe capturar en tiempo (casi) real los eventos GPS y de telemetría de cada unidad (latitud, longitud, velocidad, rumbo, timestamp) y almacenarlos en MongoDB.
RF-09	Ingesta mediante mensajería	El sistema debe permitir la ingesta de eventos de geolocalización a través de una arquitectura basada en mensajería (Apache Kafka) para desacoplar la generación y el consumo de datos.
RF-10	Historial de recorridos	El sistema debe permitir consultar el historial de recorridos de una unidad o ruta en un rango de fechas determinado.
RF-11	Cálculo de tiempos de viaje	El sistema debe calcular los tiempos de viaje reales entre paradas y compararlos contra los tiempos programados.
RF-12	Detección de atrasos	El sistema debe identificar y registrar los atrasos de las unidades respecto al horario programado, clasificándolos por severidad.
RF-13	Indicadores de desempeño	El sistema debe generar indicadores de desempeño por ruta (cumplimiento de horario, kilómetros recorridos, unidades activas, atrasos promedio).
RF-14	Data Warehouse / análisis histórico	El sistema debe consolidar en SQL Server la información histórica de movilidad para análisis y generación de reportes gerenciales.
RF-15	Proceso ETL de integración	El sistema debe ejecutar un proceso ETL que extraiga datos de PostgreSQL (rutas, choferes, unidades, horarios) y MongoDB (eventos GPS), los transforme y los cargue al Data Warehouse en SQL Server.
RF-16	Alertas por umbral	El sistema debe generar alertas automáticas cuando los datos de un sensor excedan los umbrales configurados (velocidad, temperatura del motor, geocerca, etc.).
RF-17	Búsqueda y filtrado	El sistema debe permitir la búsqueda y filtrado de unidades, choferes y rutas por estado, zona o disponibilidad.
RF-18	Actualización y baja lógica	El sistema debe permitir la actualización y eliminación lógica (no física) de registros de rutas, choferes y unidades.
RF-19	Exportación de reportes	El sistema debe permitir exportar los reportes oficiales en formato XML para su envío a entes reguladores.
RF-20	Autenticación y roles	El sistema debe permitir la autenticación de usuarios y la autorización de acciones según roles (administrador, operador, analista, chofer).

2.2 Requisitos No Funcionales

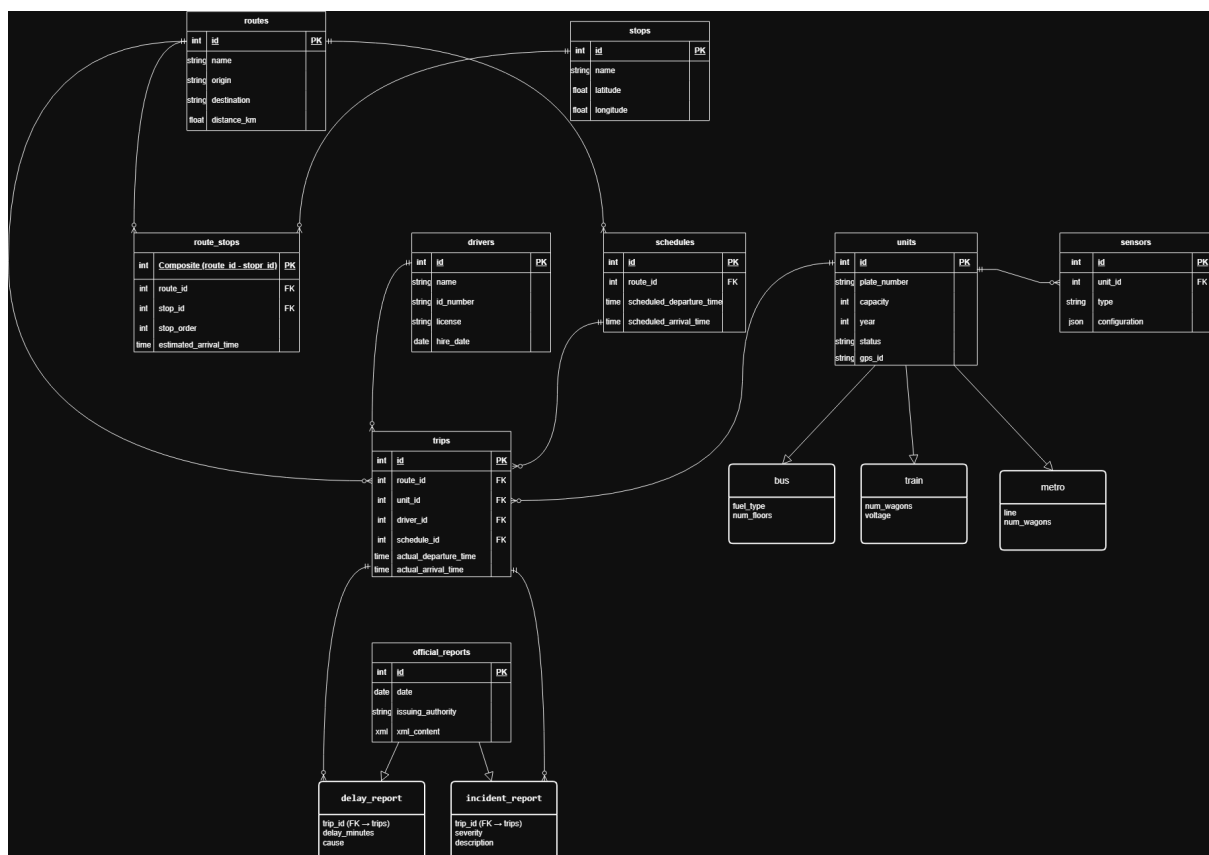
ID	Nombre	Descripción
RNF-01	Disponibilidad	El sistema debe garantizar una disponibilidad mínima del 99.5%, soportada mediante mecanismos de alta disponibilidad (réplicas/mirroring) en las bases de datos críticas (PostgreSQL, SQL Server y MongoDB).
RNF-02	Rendimiento	El sistema debe procesar y almacenar los eventos de geolocalización con una latencia menor a 5 segundos desde su generación hasta su disponibilidad para consulta.
RNF-03	Escalabilidad	El sistema debe soportar el crecimiento del número de unidades, sensores y rutas sin degradación significativa del rendimiento, permitiendo escalado horizontal en los componentes de ingesta (Kafka) y almacenamiento (MongoDB).
RNF-04	Seguridad	El sistema debe cifrar la información sensible en tránsito y en reposo, y aplicar control de acceso basado en roles a nivel de aplicación y base de datos.
RNF-05	Interoperabilidad y mantenibilidad	La arquitectura debe permitir la integración entre motores de bases de datos heterogéneos (PostgreSQL, MongoDB, SQL Server) mediante procesos ETL documentados y desacoplados, facilitando el mantenimiento evolutivo del sistema.

3. Modelo Entidad-Relación por Base de Datos

Esta sección presenta el modelo ER correspondiente a cada motor de base de datos utilizado en la solución. Los diagramas fueron elaborados por cada responsable según se indica en la bitácora (sección 6).

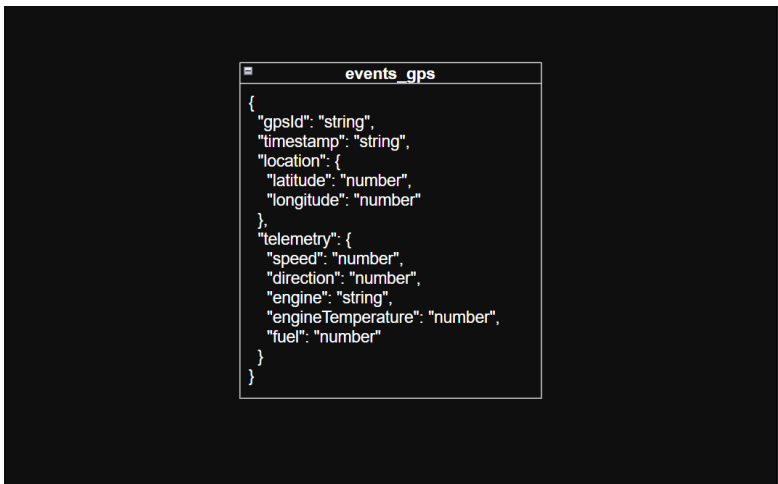
3.1 PostgreSQL - Esteban

Entidades principales: Rutas, Choferes, Horarios, Unidades. Incluye atributos JSON para configuración de sensores.



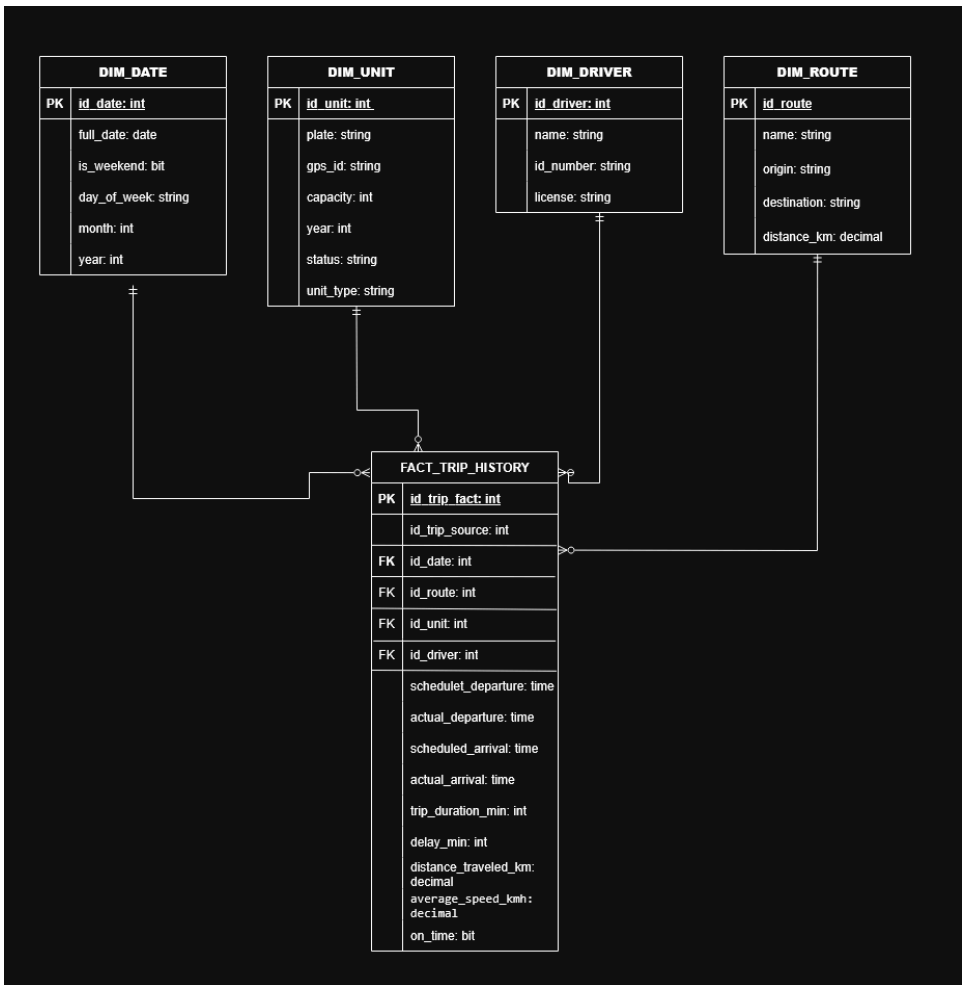
3.2 MongoDB - Laura

Colecciones principales: Eventos GPS y telemetría.



3.3 SQL Server - Jorjan

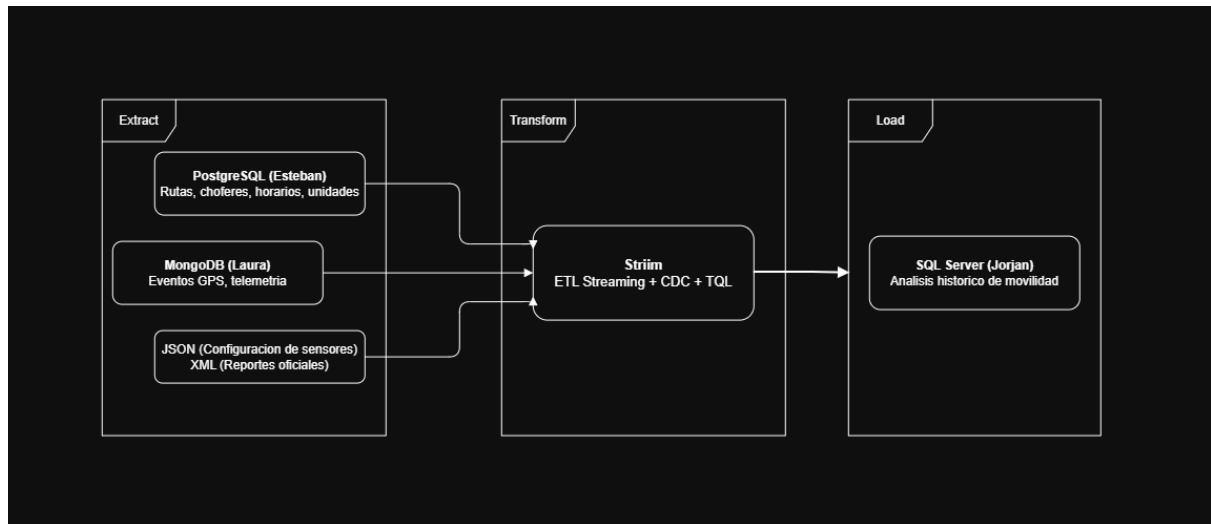
Modelo de Data Warehouse para análisis histórico de movilidad.



4. Arquitectura Propuesta

La arquitectura on-premise contempla instancias independientes por motor de base de datos, mecanismos de alta disponibilidad y un flujo de integración ETL hacia el Data Warehouse.

4.1 Diagrama de Arquitectura



4.2 Instancias de Bases de Datos

- Instancia PostgreSQL: datos maestros y operativos (Rutas, Choferes, Horarios, Unidades).
- Instancia MongoDB: eventos GPS y telemetría en alto volumen.
- Instancia SQL Server: Data Warehouse para análisis histórico de movilidad.

4.3 Estrategia de Alta Disponibilidad (Mirroring / Replicación)

Nota: esto es ficticio/preliminar, solo para tener algo razonable en el entregable. En un caso real se ajustaría según lo que la empresa encargada realmente tenga disponible.

Estrategia de Alta Disponibilidad - PostgreSQL

La idea es usar Streaming Replication, que ya viene integrada en Postgres, con un esquema de 1 Primario + 1 Standby:

- Nodo Primario: recibe todas las escrituras (rutas, choferes, unidades, trips, sensores, reportes, etc.).
- Nodo Standby: va replicando en tiempo casi real lo que pasa en el primario, leyendo su WAL (el log donde Postgres anota cada cambio antes de aplicarlo). Básicamente se mantiene "un paso detrás", listo para tomar el control si hace falta.

A diferencia de Mongo, acá el cambio de primario a standby no es automático: si el primario falla, hay que promover manualmente al standby (con `pg_ctl promote`). Para el tamaño de este proyecto tiene sentido mantenerlo así de simple, sin sumar herramientas externas de más a una arquitectura que ya combina varios motores y un ETL.

Dónde estaría cada nodo (ficticio):

- El primario, en el centro de datos principal de la empresa.
- El standby, en una sede aparte, para no perder el servicio si se cae todo un sitio completo.

Modo de replicación:

- Asíncrona, para no sacrificar el rendimiento del primario en cada escritura.

Y como respaldo extra:

- Backup diario con `pg_dump`, guardado fuera del clúster - la replicación protege contra la caída de un nodo, pero no contra errores humanos (un DELETE mal hecho se replica igual al standby).

Estrategia de alta disponibilidad - MongoDB

La idea es armar un Replica Set de 3 nodos:

- 1 nodo primario: es el que recibe todas las escrituras de los eventos GPS.
- 2 nodos secundarios: van copiando lo que pasa en el primario de forma casi en tiempo real (replicación asíncrona).

Si el primario se cae, Mongo no se queda esperando a que alguien lo arregle, inmediatamente los secundarios "votan" entre ellos y en pocos segundos uno se convierte en el nuevo primario, todo automático.

Dónde estaría cada nodo (ficticio):

- El primario y un secundario, en el centro de datos principal de la municipalidad.
- El otro secundario, en una sede aparte, para que si se cae todo un sitio completo, no se pierda la disponibilidad del servicio.

Y como respaldo extra:

- Un backup diario con [mongodump](#), guardado fuera del clúster. Esto es porque la replicación nos salva si se cae un nodo, pero no si alguien borra algo por error o los datos se corrompen para eso sirve el backup aparte.

Estrategia de alta disponibilidad - SQL-Server

La idea sería utilizar Always On Availability Groups, que es el mecanismo nativo de SQL-server para la alta disponibilidad, se plantearía configurarlo por medio de tres réplicas que serían distribuidas según su función. Las réplicas son las siguientes:

- 1 réplica primaria: es la que se encargaría de recibir todas las escrituras, ahí es donde el ETL (Striim) inserta los datos ya procesados hacia FACT_TRIP_HISTORY y actualiza las dimensiones.
- 1 réplica secundaria síncrona: se mantendría actualizada en tiempo real (commit síncrono), que estaría lista para tomar el rol de primaria, todo esto sin pérdida de datos, si la primera llegara a fallar.
- 1 réplica secundaria asíncrona, para reportes: además de funcionar como una forma de tener un respaldo adicional, podría llegar a configurarse como una réplica de solo lectura, así los analistas podrían correr consultas bastante pesadas sin tener la necesidad de competir por los recursos con las cargas del ETL.

Si la primera réplica llega a fallar, un Listener de Availability Group detecta la caída y dirige de manera automática el tráfico hacia la réplica secundaria síncrona, que se promovería a primaria. El failover es de manera automática, sin intervención alguna de manera manual, todo esto se desarrolla en cuestión de segundos.

¿Dónde se encontraría cada nodo o réplica? (ficticio)

- La primera réplica y la réplica síncrona, estarían en el centro de datos principal de la empresa
- La réplica asíncrona de solo lectura, estaría en la sede secundaria.

Respaldo independiente:

Un respaldo extra que se podría implementar es la realización de un Backup completo semanal más Backups diferenciales diarios, aplicando el mismo método utilizado en Mongo, esto para prevenir la caída de un nodo y proteger la replicación. En SQL Server esto se implementa de forma nativa por medio del comando “BACKUP DATABASE”, sin la necesidad de herramientas externas.

4.4 Integración entre Bases de Datos (ETL) - Marco

Tecnología ETL seleccionada:

Nosotros elegimos una herramienta llamada **Striim** para la integración de datos para nuestro proyecto. Utilizaremos esta herramienta por las siguientes razones:

1. Especialización en tiempo real: Striim está diseñado específicamente para streaming ETL, lo cual es indispensable dado que el sistema debe procesar eventos de geolocalización (GPS) que llegan de forma continua y no pueden esperar a un ciclo de carga nocturno.
2. Change Data Capture (CDC) nativo: Esto permite detectar inserciones y actualizaciones sin ejecutar consultas repetidas contra las bases operacionales, evitando así impactar su rendimiento.
3. TQL (Tungsten Query Language): motor de transformación con sintaxis tipo SQL que permite escribir joins entre streams, ventanas de tiempo y agregaciones, necesario para cruzar los eventos GPS con los horarios programados y calcular atrasos en el momento en que ocurren.
4. Conectividad directa a SQL Server: cuenta con conectores nativos (sink) hacia SQL Server, eliminando la necesidad de un paso intermedio adicional de carga.

Descripción del flujo:

Extract: se obtienen datos de PostgreSQL, MongoDB y archivos JSON/XML. Striim captura los cambios de las bases de datos mediante mecanismos de Change Data Capture (CDC) y realiza la ingestión de los archivos JSON/XML.

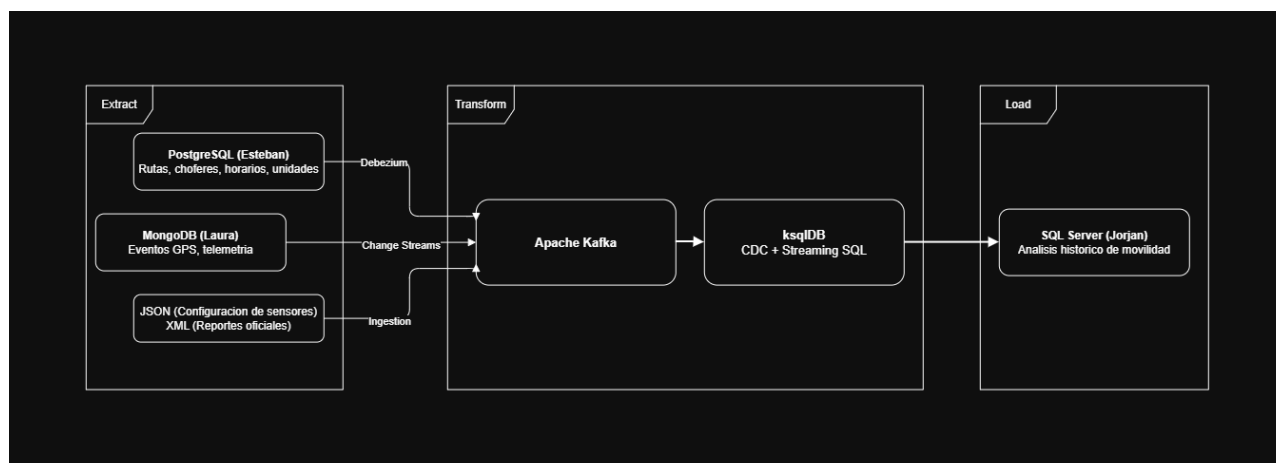
Transform: Striim procesa y transforma los datos mediante TQL (Tungsten Query Language), permitiendo cruzar los eventos GPS con los horarios programados, calcular atrasos, enriquecer la información y generar métricas por ruta, chofer y unidad en tiempo real.

Load: los datos procesados por Striim se cargan directamente en SQL Server, donde alimentan las tablas del Data Warehouse para realizar el análisis histórico de movilidad.

Opción de investigación: Apache Kafka

Apache Kafka se propone como capa de ingesta para los eventos de geolocalización en tiempo casi real. Cada unidad publicaría su posición GPS en un topic de Kafka apenas se genera el dato, desde ahí, un motor de streaming (ksqlDB) cruzaría esos eventos con los horarios programados para calcular atrasos, y un conector JDBC cargaría el resultado en SQL Server.

Apache Kafka se utilizaría como capa intermedia entre las unidades de transporte y MongoDB, no como reemplazo de Striim. Las unidades publicarían sus coordenadas a un topic de Kafka en lugar de escribir directo a MongoDB, un proceso consumidor persistiría esos eventos en MongoDB tal como lo requiere el diseño actual, y Striim seguiría siendo el motor de transformación y carga hacia SQL Server, sin cambios en su rol. Así, Kafka funciona como un buffer adicional útil si el volumen de unidades y frecuencia de reporte crece más allá de lo que la arquitectura actual puede manejar directamente.



5. Scripts DDL y CRUD

Los scripts de creación de estructura (DDL) y de operaciones CRUD de cada base de datos no se incluyen en este documento; se referencian mediante enlace al repositorio correspondiente.

ID	Nombre	Descripción
PostgreSQL	Esteban	script postgres
MongoDB	Laura	script mongo
SQL Server	Jorjan	script sql sever

6. Bitácora de Actividades

A continuación se registra, de forma general, el detalle de las actividades realizadas por el equipo a lo largo del proyecto. La columna "Integrante(s)" indica quién(es) participaron en cada actividad, considerando que en algunos casos hubo colaboración entre integrantes (por ejemplo, Esteban y Laura se apoyaron mutuamente en tareas puntuales).

Fecha	Integrante(s)	Actividad	Detalle	Horas
01/07/2026	Todo el equipo	Análisis del escenario y definición de alcance	Revisión del enunciado del Escenario 3 (Transporte Público Inteligente) y distribución de responsabilidades por motor de base de datos.	2
03/07/2026	Esteban	Diseño del modelo ER de PostgreSQL	Definición de entidades Rutas, Choferes, Horarios y Unidades, incluyendo el atributo JSON de configuración de sensores.	4
03/07/2026	Laura	Diseño del modelo de documentos de MongoDB	Estructura de la colección de eventos GPS y telemetría (esquema de documentos e índices).	4
05/07/2026	Esteban y Laura	Revisión conjunta PostgreSQL - MongoDB	Validación de las referencias entre unidades (PostgreSQL) y eventos GPS (MongoDB) para asegurar consistencia en el ETL.	3
07/07/2026	Jorjan	Diseño del Data Warehouse en SQL Server	Modelado de hechos y dimensiones para el análisis histórico de movilidad.	4

08/07/2026	Marco	Diseño del proceso ETL	Definición del flujo de extracción, transformación y carga desde PostgreSQL y MongoDB hacia SQL Server; investigación de Apache Kafka para ingesta de eventos.	4
10/07/2026	Esteban	Elaboración de script DDL/CRUD PostgreSQL	Creación de tablas, relaciones y carga de datos de prueba.	3
10/07/2026	Laura	Elaboración de script CRUD MongoDB	Operaciones de inserción y consulta de eventos GPS de prueba.	3
11/07/2026	Jorjan	Elaboración de script DDL/CRUD SQL Server	Creación de tablas del Data Warehouse y procedimientos de carga.	3
13/07/2026	Todo el equipo	Elaboración del documento final	Redacción de requerimientos, diagramas de arquitectura, estrategia de mirroring/ETL y consolidación de la bitácora.	3